

Tema 5

Métodos



Índice

- Introducción
- Sintaxis de métodos en C#
- Parámetros
- Paso de parámetros por valor y por referencia
- Valores de retorno
- Sobrecarga de métodos
- Métodos recursivos
- Prácticas recomendadas para escribir métodos



¿Qué son los métodos?

- Un **método** es un bloque de código con instrucciones que puede ser invocado o llamado desde otras partes de un programa.
- Los métodos son fundamentales en la **programación estructurada** y la **orientación a objetos**, ya que permiten encapsular la lógica de un programa en unidades manejables y reutilizables.

1. Introducción



- En C#, **todas las instrucciones** se ejecutan en el contexto de un método.
- El método `Main` es el **método de entrada** para todos los programas en C#. Se invoca desde el CLR cuando inicia el programa y comienza la ejecución del código escrito por nosotros a partir de dicho método.

1. Introducción

Características

- Los métodos evitan que tengamos que repetir código que realiza la misma función múltiples veces, permitiendo así la **reutilización**.
- Los métodos pueden ser llamados o **invocados** tantas veces como sea necesario implementando una única vez el comportamiento.
- Al reducir así el código escrito, **reducimos** con ello la probabilidad de cometer **errores**, tanto de codificación como de comportamiento.

1. Introducción

- También facilita el seguimiento del flujo y la detección de errores al **centralizar** el comportamiento y no tenerlo repetido numerosas veces.
- Los métodos además **encapsulan** la lógica de los programas, troceándolos en unidades manejables y reutilizables.
- Esta encapsulación nos permite **modularizar** las aplicaciones, clases o los propios métodos

1. Introducción

- Nos permiten **estructurar** la programación y tener un flujo de control distinto a simplemente ejecutar las instrucciones en orden secuencial (*programación secuencial*, paradigma "primitivo" de programación).
- Cuando un programa crece en tamaño y complejidad, se vuelve imprescindible abordar su complejidad dividiéndola.
- Este enfoque, conocido en la programación como **divide y vencerás** consiste en dividir un problema complejo en otros más pequeños y sencillos tantas veces como haga falta.

1. Introducción

Ejemplo

- Queremos realizar una transferencia bancaria en la app de un banco.
- Realizar el proceso completo es una tarea muy compleja que conlleva más operaciones de las que pueda parecer en un principio.
- Por ello, dividimos el problema inicial en subtareas más pequeñas y sencillas:

1. Introducción

- Mostrar la pantalla de inicio de sesión.
- Comprobar que las credenciales sean válidas.
- Cargar la información del usuario que acaba de iniciar sesión.
- Mostrar la pantalla de realizar transferencia.
- Comprobar que se cumplen los requisitos (hay dinero en la cuenta, la cuenta destino es válida, está especificado un concepto y un destinatario...)
- Realizar la transferencia.
- Mostrar la pantalla de información de la transferencia realizada.

1. Introducción

- Como vemos, una tarea aparentemente atómica, se subdivide en numerosas tareas más sencillas.
- Además, cada una de estas subtareas es susceptible de volver a dividirse nuevamente en otras más pequeñas.
- Los **métodos** nos ayudan a realizar esta división en subtareas sencillas.

1. Introducción

- La sintaxis básica de los métodos en C# es la siguiente:

- Ejemplo:

2. Sintaxis de métodos en C#

```
modificador_de_acceso tipo_de_retorno NombreMetodo(tipo_parametro  
{  
    // Cuerpo del método
```

```
1 public double Percentage(double quantity, double  
2 {  
3     return quantity * percentage / 100;
```

Modificador de acceso

- El modificador de acceso proporciona control sobre qué partes del código van a poder invocar un método.
- Podemos declarar métodos públicos (`public`, cualquier código puede invocar el método), privados (`private`, sólo el código declarado en la misma clase puede acceder al método), internos (`internal`, sólo puede invocar al método código perteneciente al mismo ensamblado)...

2. Sintaxis de métodos en C#

Modificador de acceso

- Entraremos en detalle sobre los modificadores de acceso en el tema 7 (clases y objetos).

****Nota:** en la clase Program.cs además del modificador de acceso debemos especificar obligatoriamente el modificador **static**. Veremos en el tema 7 que significa.*

2. Sintaxis de métodos en C#

Tipo de retorno

- En C#, los métodos pueden devolver o no un valor al autor de la llamada.
- Este tipo de retorno puede ser cualquier tipo de dato. El método debe finalizar con una sentencia `return` seguido de una variable, constante o expresión que coincida con el tipo de valor de retorno.
- En caso de no devolver ningún valor, el método será `void`.

2. Sintaxis de métodos en C#

Nombre

- Cada método debe tener un nombre válido.
- El nombre forma parte de la firma de un método (cómo se identifica a nivel interno), pero no es lo único que se tiene en cuenta.
- La firma también tiene en cuenta el ámbito donde está definido (clase) y sus parámetros.

2. Sintaxis de métodos en C#

Nombre

- Así pues, pueden existir varios métodos con el mismo nombre si cumplen alguno de los siguientes requisitos:
 - Pertenecen a distinta clase.
 - Tienen diferentes lista de parámetros.

2. Sintaxis de métodos en C#

Parámetros

- Los parámetros de un método son valores de entrada que se les pasa en la invocación.
- En C#, los parámetros se definen entre paréntesis después del nombre, teniendo que especificar antes el tipo de cada parámetro y separándolos con comas si hay varios.
- En el siguiente punto vamos a ver los parámetros detenidamente.

2. Sintaxis de métodos en C#

- Los parámetros son, pues, **variables** que se definen en la firma del método.
- El código del método tendrá acceso a dichas variables y podrá realizar operaciones con ellas.
- Los parámetros son siempre **opcionales**, un método puede no tener parámetros.

3. Parámetros

- En ese caso, después del nombre del método, se ponen paréntesis vacíos ().
- Normalmente, los métodos sin parámetros los utilizaremos para modificar el estado interno de un objeto o para mostrar información
- Los métodos con parámetros los utilizaremos habitualmente cuando tengamos que hacer operaciones con dichos parámetros.

3. Parámetros

Parámetros y argumentos

- Cuando invocamos un método debemos especificar el valor de los parámetros en la llamada.
- A este valor de los parámetros, o la instancia de las variables que definen los parámetros los llamamos **argumentos**.
- Así pues, el parámetro es la definición genérica y el argumento la instancia con la que vamos a llamar un método en un punto concreto del código.

3. Parámetros

Ejemplo

- En este ejemplo, `dividend` y `divisor` son los parámetros, ambos de tipo `double`.
- Cuando llamamos a ese método desde otras partes del código, debemos pasar **argumentos** a la llamada. Estos pueden ser valores, variables o expresiones.

3. Parámetros

```
1 public double Divide(double dividend, double divisor) {  
2     {  
3         return dividend / divisor;  
}
```

```
1 double result = Divide(186.15, 5.68);
2
3 double divideArea = Divide(area, 2.5);
4
5
```

Parámetros por defecto

- En C#, podemos definir **parámetros por defecto**. Un parámetro por defecto es aquel al que se le asigna un valor predeterminado en la firma del método.
- Si no se pasa un valor para ese parámetro cuando se llama al método, utilizará el valor predeterminado que hemos especificado.
- Para declarar un parámetro por defecto, simplemente se le asigna un valor al parámetro en la definición del método.

3. Parámetros

Ejemplo

- Este ejemplo define un parámetro por defecto (domain). Si cuando se realice la llamada al método no se indica valor para dicho parámetro, este tomará el valor especificado en la declaración (en este caso "gmail.com").

3. Parámetros

```
1 public void PrintEmail(string username, string domain =  
2 {  
3     Console.WriteLine(string.Format("{0}@{1}", username,  
4  
1 PrintEmail("cursoc13", "hotmail.com")  
2 // Imprime 'cursoc13@hotmail.com'  
3  
4 PrintEmail("cursoc13");  
5
```

- Los parámetros por defecto siempre tienen que ir **al final** de la lista de parámetros.
- Los parámetros por defecto son **opcionales**.
- Se puede definir más de un parámetro por defecto en un método (pero siempre tienen que ir al final en la lista).

3. Parámetros

Ejemplo

3. Parámetros

```
1 public void SendEmail(string recipient,  
2                       string subject = "(Sin Asunto)",  
3                       string body = string.Empty)  
4 {  
5     Console.WriteLine("Enviando email a: " + recipient);  
6     Console.WriteLine("Asunto: " + subject);  
7     Console.WriteLine("Cuerpo: " + body);  
8 }  
- -  
  
1 SendEmail("antonio@mail.com");  
2 // Enviando email a: antonio@mail.com  
3 // Asunto: (Sin Asunto)  
4 // Cuerpo:  
5  
6 SendEmail("maria@mail.com", "Reunión");  
7 // Enviando email a: maria@mail.com  
8 // Asunto: Reunión  
9 // Cuerpo:  
10  
11 SendEmail("alberto@mail.com", "Informe", "Por favor, revisa el informe");  
12 // Enviando email a: alberto@mail.com  
13 // Asunto: Informe
```

- En el tema anterior vimos los conceptos de tipos de valor y tipos de referencia.
- En los métodos, los parámetros siguen los mismos principios, existe el paso de parámetros por valor para los tipos de valor (por defecto) y el paso de parámetros por referencia para los tipos de referencia.

4. Paso de parámetros por valor y por referencia

- Los parámetros de tipo de **valor** reciben una copia de la variable. Cualquier modificación realizada dentro del método no afecta la variable original.
- Los parámetros de tipo de **referencia** reciben la referencia de la variable. Cualquier modificación dentro del método modifica la variable original.

4. Paso de parámetros por valor y por referencia

Ejemplo

- Al pasar un `int`, de tipo valor, aunque el método modifique el valor del parámetro, este parámetro es una copia y la variable original **no se ve afectada**.

4. Paso de parámetros por valor y por referencia

```
1 public int Add(int baseNumber, int  
2 {  
3     baseNumber += addition;  
4     return baseNumber;  
5 }
```

```
1 int number = 6;  
2 Add(number, 4);  
3  
4 Console.WriteLine(nu
```

Ejemplo

- Al pasar un tipo de referencia, cualquier modificación dentro del método **modifica la variable original**.

4. Paso de parámetros por valor y por referencia

```
1 public void ChangeModel(Car car, string n
2 {
3     car.Model = newModel;
```

```
1 Car leon = new Car("Seat", "León", 2024);
2 leon.ChangeModel("Ibiza");
3
```

```
4 Console.WriteLine("El coche es un " + leon.Brand + " ")
```

- Si deseamos modificar el contenido de una variable de tipo de valor dentro de un método, podemos especificar que se pase como referencia.
- Para ello utilizamos la palabra clave `ref` antes del tipo de parámetro:

4. Paso de parámetros por valor y por referencia

```
1 public void Add(ref int baseNumber, int  
2 {  
3     baseNumber += addition;  
4 }
```

- En este caso, el parámetro `baseNumber` se está pasando **por referencia**.
- Esto implica que cualquier modificación dentro del método afectará a la variable original pasada como argumento:
- Al pasar la variable `number` por referencia, el método ha modificado su contenido.

4. Paso de parámetros por valor y por referencia

```
1 int number = 6;  
2 Add(ref number, 4);  
3  
4 Console.WriteLine(nu
```

- En el caso de que un parámetro esté especificado como ref, el argumento en la llamada debe ser siempre una **variable asignada**, no se permiten variables declaradas sin asignar, constantes o expresiones en este caso.
- En la llamada utilizaremos igualmente la palabra clave ref antes del nombre de la variable (ver diapositiva anterior).

4. Paso de parámetros por valor y por referencia

- También se puede utilizar la palabra clave out.
- Esta palabra clave significa que el método va a asignar un valor **obligatoriamente** a la variable pasada como parámetro (con ref no es obligatorio).
- La variable puede no estar inicializada cuando se pasa con out, basta con que esté declarada.

4. Paso de parámetros por valor y por referencia

Ejemplo

- En este caso, pasamos una variable sin inicializar, únicamente declarada (today). El método asigna un valor a dicha variable y por tanto modifica la original.

4. Paso de parámetros por valor y por referencia

```
1 public void SetTodayDay(out Day(  
2 {  
3     day = DateTime.Now.DayOfWeel  
4 }
```

```
1 DayOfWeek today;
```

```
2 SetTodayDay(out today);  
3  
4 Console.WriteLine(today);
```

- Como hemos visto en el punto 2, los métodos pueden **devolver un valor**.
- El valor devuelto siempre tiene que ser el mismo que se especificó en la firma del método.
- El tipo del valor devuelto puede ser cualquiera, ya sea tipos de datos básicos (`int`, `double`, `bool`, ...) o tipos de datos personalizados.
- Los métodos pueden no devolver valor (en ese caso son de tipo `void`).

5. Valores de retorno

Devolver un valor

- Para devolver un valor utilizamos la palabra clave `return` seguido de una constante, variable o expresión que **coincidan con el tipo de retorno**.
- El ejemplo anterior daría error ya que el método se define como `double` y está intentando devolver un `string`.

5. Valores de retorno

```
1 public double CalculatePercentage(double number, c
2 {
3     return "El porcentaje es: " + number * percent
4     // ERROR. No compila.
```

- **Todas las rutas** del código deben devolver un valor.
- Si utilizamos estructuras de control como `if-else` o `switch`, debemos asegurarnos de que todos los posibles caminos devuelvan un valor. De lo contrario, resultará en un **error** y no se podrá compilar:

5. Valores de retorno

```
1 public double Divide(double dividend, double divisor)
2 {
3     if(divisor != 0)
4     {
5         return dividend / divisor;
6     }
7 }
```

- El ejemplo anterior resulta en un **error**, ya que en el caso de que el parámetro `divisor` sea 0 no hay ninguna sentencia `return` que devuelva un valor.
- Debemos asegurarnos de que todos los posibles caminos devuelvan un valor del tipo de retorno:

5. Valores de retorno

```
1 public double Divide(double dividend, doubl
2 {
3     if(divisor != 0)
4     {
5         return dividend / divisor;
6     }
7     else
8     {
9         return double.NaN;
10 }
```

- En el caso de los switch-case, si los casos devuelven un valor, es obligatorio poner el caso default que devuelva también un valor (o poner un return posterior fuera del switch-case).

5. Valores de retorno

```
1 public int GetValue(int option)
2 {
3     switch(option)
4     {
5         case 1:
6             return 125;
7         case 2:
8             return 280;
9         case 3:
10            return 742;
11        default:
12            return 0;
13        // Si no especificamos el c
14        // default, da error al com
15    }
```

Recoger un valor devuelto

- Para recoger el valor devuelto por un método, debemos asignar la llamada a una variable del mismo tipo de dato que el valor de retorno del método.
- No es obligatorio hacerlo, pero si no lo hacemos se pierde el valor devuelto.

5. Valores de retorno

```
1 CalculatePercentage(80, 33.3);  
2 // El resultado de la operación se pierde  
3  
4 double percentage = CalculatePercentage(50, 25);  
5 // La variable 'percentage' almacena 12.5 tras la  
6  
7 string pctString = CalculatePercentage(48, 8.4);  
8 // El resultado de la operación se pierde
```

Métodos void

- Los métodos que no devuelven un valor son de tipo void.
- En estos métodos, se puede utilizar la sentencia return para finalizar su ejecución, pero no puede ir seguido de ningún valor.

5. Valores de retorno

Ejemplo

- El método del ejemplo comprueba si el parámetro `car` es `null`. Si lo es finaliza el método con la instrucción `return`. Si no lo es, el método continua su ejecución.

5. Valores de retorno

```
1 public void PrintCarInfo(Car car)
2 {
3     if(car == null)
4     {
5         return;
6     }
7 }
```

```
8 Console.WriteLine("Marca: " + car  
9 Console.WriteLine("Modelo: " + ca  
10 Console.WriteLine("Año: " + car.V
```

Vídeo práctico

5. Valores de retorno

Vídeo práctico - código

5. Valores de retorno

```
1  static void Main(string[] args)
2  {
3      PrintGreetings("Juan");
4      Console.WriteLine();
5
6      int squareNumber = Square(5);
7      string squareNumber2 = SquareString(5);
8
9      Console.WriteLine(squareNumber);
10     Console.WriteLine(squareNumber2);
11
12     double number = 10;
13     Half(ref number);
14
15     Console.WriteLine(number);
16     Console.WriteLine();
17
18     Console.WriteLine(GenerateAddress(5, "C/ Mayor",
19     Console.WriteLine();
20     Console.WriteLine(GenerateAddress(25, "Hilster St
```

- La **sobrecarga de métodos** es una característica fundamental en C# que nos permite definir múltiples métodos con el **mismo nombre**, siempre que tengan **diferentes firmas**.
- La sobrecarga es un recurso útil porque mejora la legibilidad y flexibilidad del código, al permitir que diferentes variaciones de un método realicen tareas similares, pero con entradas diferentes.

6. Sobrecarga de métodos

- La firma de un método incluye el número, tipo y orden de los parámetros que recibe, pero **no su tipo de retorno**.
- Esto quiere decir que podemos sobrecargar un método que difiera en:
 - El número de parámetros
 - El tipo de parámetros
 - El orden de los parámetros
- Pero **NO** podemos sobrecargar un método con los mismos parámetros pero devuelva un tipo distinto.

6. Sobrecarga de métodos

Ejemplo

- Podemos sobrecargar un método que tiene diferente número, tipo u orden de parámetros.

6. Sobrecarga de métodos

```
1 public int Add(int a, int b)
2 {
3     return a + b;
4 }
5
6 public int Add(int a, int b,
```

```
7 {  
8     return a + b + c;
```

```
1 public void PrintPhone(string name, long phone)  
2 {  
3     Console.WriteLine("El número de {0} es {1}",  
4 }  
5  
6 public void PrintPhone(long phone, string name)  
7 {  
8     Console.WriteLine("El número de {0} es {1}",
```

Ejemplo

- **NO** podemos sobrecargar un método que tiene los mismos parámetros (aunque tengan diferente nombre o el tipo de retorno sea distinto).

6. Sobrecarga de métodos

```
1 public int Add(int a, int b)
2 {
3     return a + b;
4 }
```

```
5
6 public double Add(int a, int
7 {
8     // ERROR, coinciden los p
9     return a + b;

1 public void PrintPhone(string name, long phone)
2 {
3     Console.WriteLine("El número de {0} es {1}", name, phone)
4 }
5
6 public void PrintPhone(string username, long telephoneNumber)
7 {
8     // ERROR, coinciden los parámetros
9     Console.WriteLine("El número de {0} es {1}", username, te
```

- Cuando tengamos un método sobrecargado, los parámetros de la llamada definirán a qué método se llama (de manera automática, no tendremos que elegir "a mano").

6. Sobrecarga de métodos

```
1  public int Multiply(int a, int b)
2  {
3      return a * b;
4  }
5
6  public double Multiply(double a, double b
7  {
8      return a * b;
9  }
10
11 public double Multiply(double a, double b
12 {
13     return a * b * c;
14 }
```

- En el ejemplo anterior tenemos un método con 3 sobrecargas diferentes.
- Los parámetros de las llamadas definirán a qué versión del método se llama en cada caso:

6. Sobrecarga de métodos

```
1 int result1 = Multiply(5, 3);  
2 // Llama a la primera versión  
3  
4 double result2 = Multiply(7.85, 4);  
5 // Llama al a segunda versión  
6  
7 double result3 = Multiply(10.5, 4.2  
8 // llama a la tercera versión
```

- Es habitual encontrar numerosas sobrecargas de métodos en las librerías de la BCL y en librerías externas:

6. Sobrecarga de métodos



Vídeo práctico

6. Sobrecarga de métodos

Vídeo práctico - código

6. Sobrecarga de métodos

```
1  static void Main(string[] args)
2  {
3      Console.WriteLine("Hello, World!");
4
5      Console.WriteLine(Multiply(5, 2));
6      Console.WriteLine(Multiply(5, 2, 10));
7      Console.WriteLine(Multiply(5.5, 4));
8
9      Console.WriteLine(Repeat("hola mundo", 5));
10     Console.WriteLine();
11     Console.WriteLine(Repeat(5, "hola mundo"));
12 }
13
14 public static int Multiply(int a, int b)
15 {
16     return a * b;
17 }
18
19 public static int Multiply(int a, int b, int c)
20 {
21     return a * b * c;
22 }
```

Recursividad

- La **recursividad** es una técnica de programación en la que un método se llama a sí mismo para resolver un problema más complejo descomponiéndolo en subproblemas más pequeños.
- Así pues, un **método recursivo** es aquel que se invoca a sí mismo dentro de su propia definición.

7. Métodos recursivos

- Este enfoque es particularmente útil cuando un problema puede expresarse naturalmente en términos de subproblemas repetitivos.
- Esto suele ser habitual en muchos algoritmos relacionados con árboles, gráficos, matemáticas y combinatoria..

7. Métodos recursivos

Estructura de un método recursivo

- Un método recursivo consta de dos partes esenciales: **caso base** y **caso recursivo**.
- El **caso base** es la condición en la que el método deja de llamarse a sí mismo, es decir, el punto de parada de la recursión.
- Sin un caso base, la recursión continuaría indefinidamente, lo que provocaría un error de desbordamiento de pila (*stack overflow*).

7. Métodos recursivos

- El **caso recursivo** es la parte del método donde este se llama a si mismo, pero con una versión más simple del problema.
- En cada llamada recursiva, el problema se simplifica progresivamente hasta que se **alcanza el caso base**.
- Si el caso recursivo no alcanza el caso base, incurriríamos en el mismo problema de recursividad indefinida y desbordamiento de la pila.

7. Métodos recursivos

Sintaxis de un método recursivo

7. Métodos recursivos

```
tipo_de_retorno NombreMetodo(tipo_parametro parametro)
{
    if (condición_de_caso_base)
    {
        // Caso base: detener la recursión
        return valor_base;
    }
    else
    {
        // Caso recursivo: llamada al método con un pará
        return NombreMetodo(parámetro_modificado);
    }
}
```

Ejemplo: Factorial

- Uno de los ejemplos más comunes para entender la recursividad es el cálculo del **factorial** de un número.
- El factorial de un número n (denotado como $n!$) es el producto de todos los enteros positivos menores o iguales a n .

7. Métodos recursivos

- Por ejemplo:
 - $5! = 5 * 4 * 3 * 2 * 1 = 120$
- El factorial de n también puede definirse de forma **recursiva**:
 - $n! = n * (n - 1)!$
 - Caso base: $0! = 1$
- Es decir, si n es igual a 0, el método devuelve 1 (caso base).
- Si n es mayor que 0, el método se llama a sí mismo con el valor $n - 1$, y multiplica ese valor por n .

7. Métodos recursivos

Implementación:

7. Métodos recursivos

```
1  public int Factorial(int n)
2  {
3      if (n == 0)
4      {
5          return 1; // Caso base
6      }
7      else
8      {
9          return n * Factorial(n - 1); // Caso
10     }
```

Ejemplo: Fibonacci

- Otro ejemplo clásico de recursividad es la **secuencia de Fibonacci**, que se define como:
 - $F(0) = 0$
 - $F(1) = 1$
 - $F(n) = F(n-1) + F(n-2)$ para $n > 1$
- La secuencia de Fibonacci comienza con 0 y 1, y cada número subsiguiente es la suma de los dos anteriores. La secuencia es:
0, 1, 1, 2, 3, 5, 8, 13, ...

7. Métodos recursivos

Implementación:

7. Métodos recursivos

```
1 public int Fibonacci(int n)
2 {
3     if (n == 0)
4     {
5         return 0; // Caso base 1
6     }
7     else if (n == 1)
8     {
9         return 1; // Caso base 2
10    }
11    else
12    {
13        return Fibonacci(n - 1) + Fibonacci(n - 2); // Ca
14    }
```

Eficiencia y rendimiento

- Aunque los métodos recursivos son muy útiles en problemas matemáticos que se definen recursivamente, hay que tener en cuenta que pueden suponer un **problema de memoria**.
- Cada llamada recursiva añade gran cantidad de datos a la pila de la memoria. Esto puede agotar la memoria en casos donde la recursividad es muy alta, pudiendo llevar a un error de ***stack overflow***, en el que el límite de la pila es excedido.

7. Métodos recursivos

- Además, los métodos recursivos también pueden suponer un problema de **rendimiento**.
- Por ejemplo, la implementación recursiva de la secuencia de Fibonacci recalcula los mismos valores múltiples veces, lo que lleva a una **complejidad exponencial**.
- Para valores muy altos en la secuencia, el tiempo de procesamiento se hace inabordable por un ordenador personal.

7. Métodos recursivos

Vídeo práctico

7. Métodos recursivos

Vídeo práctico - código

7. Métodos recursivos

```
1  static void Main(string[] args)
2  {
3      int input = 189846516;
4
5      Console.WriteLine("La suma de los dígitos de " + input);
6
7      Stopwatch stopwatch = new Stopwatch();
8      stopwatch.Start();
9      Console.WriteLine(Fibonacci(46));
10     stopwatch.Stop();
11     Console.WriteLine("Tiempo: " + stopwatch.Elapsed);
12 }
13
14 public static int AddDigits(int number)
15 {
16     if(number == 0)
17     {
18         return 0;
19     }
20
21     int digit = number % 10;
```

- Escribir métodos de manera **eficiente y clara** es fundamental para el desarrollo de software de calidad.
- Un buen diseño de métodos facilita la comprensión, el mantenimiento y la reutilización del código, mientras que lo contrario puede resultar en código con errores y además hacer que estos sean muy difícil de encontrar.
- Vamos a ver algunos consejos y prácticas recomendadas para ello.

8. Prácticas recomendadas para escribir métodos

Nombrado de métodos

- El nombre de un método debe ser **claro y descriptivo**, de manera que indique de forma precisa lo que hace.
- Normalmente, el nombre de un método se compondrá de un **verbo** y, opcionalmente, un nombre.
- Evita utilizar nombres genéricos que no indican claramente que acción realizan.

8. Prácticas recomendadas para escribir métodos

▸ Ejemplos de nombres buenos:

- `PrintReport`
- `CreateUser`
- `SaveFile`

▸ Ejemplos de nombres malos:

- `MyMethod`
- `Method1`
- `DoTask`

8. Prácticas recomendadas para escribir métodos

Principio de Responsabilidad Única

- Según el **Principio de Responsabilidad Única** (Single Responsibility Principle, SRP), cada método debe tener una única responsabilidad o propósito. Es decir, un método debe hacer **una sola cosa** y hacerla bien.
- Si un método realiza varias tareas, se vuelve difícil de entender, probar y mantener, debemos partir el método en varios si realizamos varias tareas en uno. El nombre puede ser una pista, si contiene 2 verbos probablemente sea un indicativo de ello (ej: SaveAndPrint).

8. Prácticas recomendadas para escribir métodos

Longitud de los métodos

- Idealmente, un método no debería tener más de **20 o 30 líneas** de código. Los métodos largos son difíciles de leer y propensos a errores.
- Esto no siempre es posible, pero si tenemos métodos muy largos, son candidatos a revisar e intentar trocear en métodos más pequeños.
- Una buena práctica es simplificar expresiones cuando podamos utilizar una equivalente más corta y sencilla.

8. Prácticas recomendadas para escribir métodos

Número de parámetros

- Los métodos con muchos parámetros pueden ser **difíciles** de entender y usar.
- Si un método necesita demasiados parámetros, es posible que esté haciendo demasiado trabajo o que se deba reconsiderar el diseño.
- Se pueden encapsular varios parámetros en objetos únicos, lo veremos en el tema 7, acerca de clases y objetos.

8. Prácticas recomendadas para escribir métodos

Uso de sobrecarga y parámetros por defecto

- Si tenemos pequeñas variaciones de métodos según los parámetros de entrada, es una buena práctica escribir sobrecargas y especificar parámetros por defecto u opcionales.
- Esto permite que el código sea más limpio y fácil de mantener.

8. Prácticas recomendadas para escribir métodos

Tipos de retorno

- El tipo de retorno de un método debe ser adecuado para lo que el método está haciendo.
- Evita devolver tipos genéricos si un tipo específico es más adecuado.
- Por ejemplo, vimos que todas las clases heredan de `object` así que en cualquier método podemos devolver un tipo `object` y sería válido.

8. Prácticas recomendadas para escribir métodos

- También podemos devolver un tipo numérico diferente si hay conversión implícita a su tipo, pero no es recomendable.
- Ambos métodos son válidos, pero es recomendable declararlos de un **tipo más específico**: `double` en el primer caso (el tipo con el que trabaja el método) e `int` en el segundo (nunca hay decimales).

8. Prácticas recomendadas para escribir métodos

```
1 public object Divide(double dividend, double divisor)
2 {
3     return dividend / divisor;
```

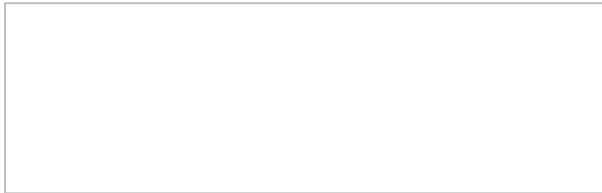
```
4 }  
5  
6 public double Add(int a, int b)  
7 {  
8     return a + b;
```

Variables externas

- A no ser que sea su objetivo, un método **nunca debe modificar** variables externas a su ámbito.
- Esta práctica puede provocar **efectos secundarios** que hacen que el código sea impredecible y muy difícil de depurar.
- Es preferible devolver un resultado en lugar de modificar el estado global de un objeto.

8. Prácticas recomendadas para escribir métodos

En este tema hemos aprendido a escribir nuestros propios métodos. Esto nos proporciona una gran flexibilidad a la hora de escribir programas y constituye una de las bases de la programación estructurada y de la orientada a objetos.



Speaker notes